

NODO TECNOLÓGICO DE CATAMARCA

Profesor: Ing. Daniel Maldonado

PROPUESTA DE CAPACITACIÓN

Módulo 1: Lógica y Fundamentos de Programación con JavaScript

Formar el pensamiento algorítmico que sostiene cualquier lenguaje, en un contexto donde la inteligencia artificial ya es una herramienta cotidiana de trabajo.

Duración	8 clases de 1h20, distribuidas en 4 semanas (2 clases semanales)
Docente propuesto	Ing. Daniel Maldonado
Días y horarios propuestos	A definir según disponibilidad del Nodo Tecnológico
Modalidad	Presencial. Clases teórico-prácticas, cada encuentro dividido en dos bloques de 40 minutos
Producto final	Proyecto integrador funcional en consola (ej. sistema de registro de ventas)
Perfil sugerido	Personas sin formación previa en programación, de perfil variado
Enfoque	Lógica de programación aplicada con JavaScript, y uso crítico de IA como apoyo de estudio

1. Fundamentación

Las herramientas de inteligencia artificial generativa permiten hoy producir código funcional en segundos. Esto ya es un hecho instalado en la forma de trabajar de cualquier programador, y no tiene sentido pretender que un alumno que recién empieza no la use. Lo que cambia es el rol del docente: pasa a ser un tutor que guía al alumno para que, aun usando IA, comprenda lo que está pasando y no se limite a copiar resultados.

Este módulo asume la IA como parte del proceso de aprendizaje desde el principio, pero con una secuencia clara: primero se fija el razonamiento propio con uno o dos ejercicios manuales por tema, y recién después se habilita el uso de IA para resolver el resto de la práctica, siempre bajo la supervisión del docente, que verifica que el alumno pueda explicar, modificar y corregir lo que la IA produjo.

El módulo está pensado para personas sin formación previa en programación. No busca que el alumno “aprenda JavaScript” de memoria, sino que incorpore la lógica de programación utilizando JavaScript como lenguaje vehicular, con una estructura de 4 semanas, 2 clases semanales de 1 hora 20 minutos, divididas internamente en dos bloques de 40 minutos cada una.

2. Objetivo general

Que los alumnos incorporen la lógica de programación estructurada (secuencia, condición y repetición) utilizando JavaScript, alcanzando la capacidad de leer, escribir y depurar programas simples, y de usar herramientas de inteligencia artificial como apoyo de estudio sin perder criterio propio sobre el código que producen.

3. Objetivos específicos

- Distinguir entre sintaxis y lógica de programación.
- Comprender de forma general el esquema de funcionamiento de la web (cliente, servidor, navegador, comunicación mediante HTTP) para situar en contexto el rol de JavaScript.
- Comprender el rol de variables, tipos de datos y operadores en la representación de información.
- Aplicar estructuras condicionales simples, dobles y múltiples para resolver problemas de decisión.
- Aplicar estructuras repetitivas utilizando contadores y acumuladores.
- Utilizar el operador ternario como forma abreviada de una decisión simple.
- Identificar y manejar errores básicos de sintaxis, de ejecución y de lógica.
- Usar herramientas de inteligencia artificial como apoyo para explicar, revisar y depurar código propio, sin delegar en ellas el razonamiento.
- Construir un proyecto integrador simple que combine todos los conceptos del módulo.

4. Destinatarios

El módulo está dirigido a personas sin experiencia previa en programación, de perfil variado (estudiantes, personas en reconversión laboral, público general interesado en tecnología), que buscan una introducción sólida a la lógica de programación como primer paso antes de especializarse en desarrollo web, backend o análisis de datos.

5. Por qué JavaScript como lenguaje de entrada

La elección de JavaScript por sobre Python para este módulo introductorio responde a un criterio concreto: qué puede mostrar el alumno como producto final al terminar la capacitación, y qué tan reutilizable es ese aprendizaje.

- **Un solo lenguaje, múltiples salidas: JavaScript es el único lenguaje que corre nativamente en el navegador, y con él el alumno puede eventualmente construir frontend web, backend (Node.js), aplicaciones móviles y de escritorio.**
- Resultado visible desde temprano: a diferencia de un lenguaje orientado a consola o backend puro, JavaScript permite ver resultados interactivos y visuales en poco tiempo, factor motivacional clave en público que recién empieza.
- Progresión natural hacia Python: quienes luego quieran especializarse en backend, datos o automatización, llegarán a Python con la lógica ya incorporada. El aprendizaje se reduce a sintaxis nueva, no a lógica nueva.

En síntesis: JavaScript funciona como puerta de entrada al pensamiento algorítmico con salida práctica inmediata; Python queda reservado como especialización posterior para quien busque profundizar en backend o datos.

6. Metodología

- Cada clase teórica se complementa con una clase práctica inmediatamente posterior, evitando la acumulación de temas sin afianzar.
- Cada encuentro de 1h20 se organiza en dos bloques de 40 minutos, alternando exposición conceptual y trabajo guiado.
- Los alumnos avanzan de forma progresiva sobre ejercicios de complejidad creciente, hasta llegar al proyecto integrador de la Clase 8.
- En cada clase práctica, el alumno resuelve primero uno o dos ejercicios de forma manual, sin asistencia de IA, para fijar el razonamiento del tema. El resto de la práctica se resuelve con apoyo de IA, bajo tutoría activa del docente.
- Modalidad presencial, con el docente como tutor del proceso: no solo corrige, sino que verifica que el alumno entienda lo que la IA generó.

Criterio de uso de IA dentro del módulo:

1. Primero lo manual, después la IA. Cada tema arranca con uno o dos ejercicios resueltos sin asistencia, para que el alumno fije el razonamiento propio antes de delegar.
2. La IA acelera, el docente tutoriza. A partir de ahí el alumno puede resolver con IA con libertad, pero el docente acompaña el proceso y pide que explique lo que produjo.
3. Todo código se prueba y se explica en clase. Propio o generado con IA: se modifica, se rompe a propósito, y se entiende qué pasó y por qué.

7. Esquema general

Semana	Clase	Tipo	Duración	Contenido
1	1	Teórica	1h20 (2 x 40')	Introducción a la programación + Variables y tipos de datos
1	2	Práctica	1h20 (2 x 40')	Aplicación de variables y tipos de datos
2	3	Teórica	1h20 (2 x 40')	Operadores, entrada/salida y operador ternario
2	4	Práctica	1h20 (2 x 40')	Aplicación de operadores
3	5	Teórica	1h20 (2 x 40')	Estructuras condicionales
3	6	Práctica	1h20 (2 x 40')	Aplicación de condicionales
4	7	Teórica	1h20 (2 x 40')	Bucles, manejo de errores y buenas prácticas
4	8	Práctica	1h20 (2 x 40')	Proyecto integrador

8. Detalle por clase

Semana 1

Clase 1 — Teórica · Semana 1 · 1h20 (2 bloques de 40 min)**Introducción a la programación + Variables y tipos de datos****Bloque 1 (40 min)**

- Qué es un algoritmo, qué es un lenguaje de programación
- Esquema general de funcionamiento de la web: cliente, servidor, navegador
- Comunicación mediante HTTP: qué es una petición y una respuesta
- JavaScript como lenguaje interpretado, dónde corre (navegador y Node.js)
- Entorno de desarrollo: consola del navegador, VSCode, Node.js
- Instalación y primer contacto con el entorno

Bloque 2 (40 min)

- Qué es una variable, concepto de memoria y almacenamiento
- Declaración de variables: let, const (por qué evitar var)
- Tipos de datos primitivos: number, string, boolean, undefined, null
- Convenciones de nomenclatura (camelCase)

Clase 2 — Práctica · Semana 1 · 1h20 (2 bloques de 40 min)**Aplicación de variables y tipos de datos****Bloque 1 (40 min)**

- Ejercitación guiada: declaración, asignación y reasignación de variables
- Verificación de tipos con console.log() y typeof

Bloque 2 (40 min)

- Casos de uso reales que involucren registro de datos simples
- Corrección grupal y despeje de dudas

Semana 2**Clase 3 — Teórica · Semana 2 · 1h20 (2 bloques de 40 min)****Operadores, entrada/salida y operador ternario****Bloque 1 (40 min)**

- Operadores aritméticos
- Operadores de comparación: == vs === (coerción de tipos)
- Operadores lógicos (&&, ||, !)
- Precedencia de operadores

Bloque 2 (40 min)

- Operador ternario (condición ? valorSiVerdadero : valorSiFalso) como forma abreviada de un if/else simple
- console.log() como herramienta principal de verificación de resultados

Clase 4 — Práctica · Semana 2 · 1h20 (2 bloques de 40 min)**Aplicación de operadores**

Bloque 1 (40 min)

- Ejercitación combinando operadores aritméticos, de comparación y lógicos

Bloque 2 (40 min)

- Reescritura de condicionales simples usando operador ternario
- Casos de uso reales que impliquen cálculos y comparaciones

Semana 3**Clase 5 — Teórica · Semana 3 · 1h20 (2 bloques de 40 min)****Estructuras condicionales****Bloque 1 (40 min)**

- Qué es una estructura de control
- Condicional simple (if), doble (if/else)

Bloque 2 (40 min)

- Condicional múltiple (else if, switch), anidamiento de condicionales
- Concepto de “truthy” y “falsy”
- Repaso del operador ternario en contexto de condicionales

Clase 6 — Práctica · Semana 3 · 1h20 (2 bloques de 40 min)**Aplicación de condicionales****Bloque 1 (40 min)**

- Ejercitación con condicionales simples, dobles y múltiples

Bloque 2 (40 min)

- Condicionales anidados y casos de uso reales con toma de decisiones
- Uso del ternario donde simplifique la lógica

Semana 4**Clase 7 — Teórica · Semana 4 · 1h20 (2 bloques de 40 min)****Bucles, manejo de errores y buenas prácticas****Bloque 1 (40 min)**

- Qué es una estructura repetitiva, bucle while y for
- Concepto de contador y acumulador
- Condición de corte, riesgo de bucle infinito, break y continue

Bloque 2 (40 min)

- Qué es un error en tiempo de ejecución vs error de lógica vs error de sintaxis
- Estructura try/catch
- Legibilidad de código: comentarios, indentación, nomenclatura clara

Clase 8 — Práctica · Semana 4 · 1h20 (2 bloques de 40 min)**Proyecto integrador****Bloque 1 (40 min)**

- Desarrollo guiado de un ejercicio integral que combine variables, operadores, condicionales, ternario y bucles

Bloque 2 (40 min)

- Incorporación de manejo básico de errores al ejercicio
- Cierre y repaso general del módulo

9. Evaluación sugerida

Criterio	Puntaje
Comprensión de variables y tipos de datos	15
Uso de operadores y operador ternario	15
Aplicación de estructuras condicionales	20
Aplicación de estructuras repetitivas (contadores y acumuladores)	20
Manejo de errores y buenas prácticas de código	15
Proyecto integrador (funcionalidad y claridad)	15
Total	100

10. Próximo paso sugerido

Una vez cerrado este módulo, el Módulo 2 continuaría con estructuras de datos (arrays, objetos) y funciones, manteniendo el mismo esquema de 4 semanas / 8 clases con formato teórica-práctica y bloques de 40 minutos. Superada esa etapa, el alumno queda en condiciones de avanzar hacia módulos de mayor nivel de diseño y arquitectura de software.